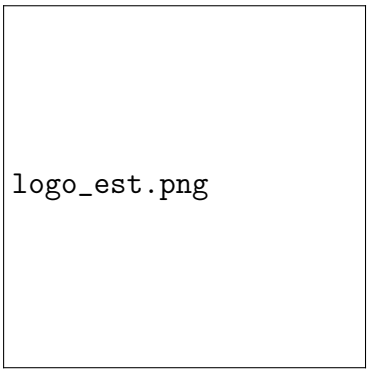


École Supérieure de Technologie

Filière Cybersécurité

Projet de Fin d'Études (PFE)

Cahier des Charges

Pipeline CI/CD Sécurisé avec Tests de Sécurité Automatisés

(SAST, DAST, SCA, IaC, Container Scanning, Secret Detection)

+ Cloud AWS, Modèle IA & Dashboard Interactif

Étudiant : **cheddad mohamed yasser/raoui abdelkamel**
Encadrant académique : M.Imane Aitouhanni
Date de soutenance : Juin 2026
Version : **2.0 (Cloud + AI + Dashboard)**

Année Universitaire 2025-2026

Table des matières

1 Contexte et Objectifs Stratégiques **3**
1.1 Contexte . . . 3
1.2 Objectif Général . . . 3
1.3 Objectifs Spécifiques . . . 3

2 Portée du Projet **4**
2.1 Inclusions . . . 4
2.2 Exclusions . . . 4

3 Architecture Technique **4**
3.1 Schéma Fonctionnel du Pipeline . . . 5
3.2 Architecture Globale . . . 5
3.3 Matrice des Outils et Technologies (Version Complète) . . . 5

4 Exigences Fonctionnelles Détaillées **6**
4.1 Application Cible (Vulnerable Web App) . . . 6
4.2 Pipeline CI/CD . . . 6
4.3 SAST (Semgrep + SonarQube) . . . 7
4.4 Secret Detection (TruffleHog) . . . 7
4.5 SCA (Snyk) . . . 7
4.6 Container Scanning (Trivy) . . . 7
4.7 DAST (OWASP ZAP) . . . 8
4.8 Modèle IA pour la Sécurité . . . 8
4.9 Déploiement Cloud AWS . . . 9
4.10 **Interface Dashboard** . . . 9
4.11 Phase de Remédiation . . . 10
4.12 Reporting . . . 10

5 Exigences Non-Fonctionnelles **10**

6 Livrables Attendus **11**

7 Planning Prévisionnel **12**

8 Critères d'Évaluation **13**

9 Risques et Mitigations **13**

10 Approbations **14**

11 Références Bibliographiques **14**

A Annexes **14**
A.1 Glossaire . . . 14
A.2 Acronymes . . . 15
A.3 Versioning du Document . . . 15
A.4 Arborescence du Projet . . . 15

Liste des tableaux

3 Exigences - Application cible 6
4 Exigences - Pipeline CI/CD 6
5 Exigences - SAST 7
6 Exigences - Secret Detection 7
7 Exigences - SCA 7
8 Exigences - Container Scanning 7
9 Exigences - DAST 8
10 Exigences - Modèle IA 8
11 Exigences - Déploiement Cloud 9
12 Exigences - Dashboard Interface 9
13 Exigences - Remédiation 10
14 Exigences - Reporting 10
17 Planning sur 9 semaines (version complète) 12
18 Grille d'évaluation (version avec Cloud, IA & Dashboard) 13
20 Approbations 14
22 Historique des versions 15

Table des figures

1 Vue d'ensemble du pipeline DevSecOps cible avec Cloud et IA 3
2 Étapes du pipeline avec security gates, Cloud AWS et IA 5
3 Architecture technique complète 5
4 Diagramme de Gantt prévisionnel (version étendue) 12

1 Contexte et Objectifs Stratégiques

1.1 Contexte

Les organisations adoptant les méthodologies DevOps intègrent la sécurité tardivement, souvent après la phase de déploiement. Cette approche expose les applications critiques à des vulnérabilités exploitables en production. L'intégration de la sécurité de manière précoce et automatisée (« Shift Left ») est devenue une exigence incontournable.

Ce projet intègre également des composants avancés :

— Déploiement cloud réel (AWS) pour simuler un environnement de production

— Modèle d'intelligence artificielle pour réduire les faux positifs et prioriser les vulnérabilités

— **Interface dashboard interactive** pour visualiser l'état du pipeline et des vulnérabilités



FIGURE 1 – Vue d'ensemble du pipeline DevSecOps cible avec Cloud et IA

1.2 Objectif Général

Concevoir, implémenter et valider un pipeline CI/CD intégrant des mécanismes de tests de sécurité automatisés (SAST, DAST, SCA, IaC, Container Registry, Secret Detection) capables de bloquer automatiquement le déploiement de code vulnérable, avec déploiement sur **AWS**, filtrage des vulnérabilités par **IA**, et visualisation via un **dashboard interactif**.

1.3 Objectifs Spécifiques

- Automatiser la détection de vulnérabilités à chaque étape du cycle de vie de développement
- Établir des « Security Gates » (seuils de qualité) basés sur des scores de risque
- Produire des rapports de conformité exploitables
- Démontrer la capacité à corriger les vulnérabilités détectées (phase de remédiation)
- Déployer l'application sur AWS (ECS Fargate) via Terraform
- Intégrer un modèle IA pour réduire les faux positifs (objectif : 30% de réduction)
- **Développer une interface dashboard pour visualiser l'état du pipeline**

2 Portée du Projet

2.1 Inclusions

Composant	Description
Application cible	Application web vulnérable OWASP Juice Shop ou application personnalisée en Python/Flask
Pipeline CI/CD	GitHub Actions (ou GitLab CI) avec configuration as code
SAST	SonarQube (Community Edition) + Semgrep (règles personnalisées)
DAST	OWASP ZAP (mode actif et passif)
SCA	Snyk (ou OWASP Dependency-Check)
IaC Scanning	Checkov (scan des Dockerfiles et configurations Terraform)
Container Scanning	Trivy (scan des images Docker avant push vers registry)
Secret Detection	TruffleHog (scan des commits et historique Git)
Cloud Platform	AWS (ECR, ECS Fargate, RDS, VPC) provisionné avec Terraform
AI Security Model	Modèle de classification (scikit-learn) pour filtrer les faux positifs SAST
AI Explainability	SHAP/LIME pour expliquer les décisions du modèle
Dashboard	**Interface React + FastAPI pour visualisation en temps réel**
Remédiation	Création de branches de correctifs, PR automatisées, validation post-fix
Reporting	Génération de rapports JUnit/Allure avec dashboards personnalisés

2.2 Exclusions

Déploiement sur cloud public — **DÉSORMAIS INCLUS (AWS)**

Tests de pénétration manuels

Conformité réglementaire (RGPD, PCI-DSS) hors scope

Entraînement d'un modèle IA générique (utilisation d'un modèle pré-entraîné ou fine-tuning léger uniquement)

3 Architecture Technique

3.1 Schéma Fonctionnel du Pipeline

Étape	Action	Décision
Code Push	Détection des changements	Déclenchement pipeline
Secret Detection	TruffleHog scan	Échec si secret trouvé
SAST	Semgrep + SonarQube	Échec si $\geq$ 1 Critical
AI Filter	Classification des vulnérabilités	Filtrage des faux positifs (30%+)
SCA	Snyk / Dependency-Check	Échec si $\geq$ 1 Critical ou 3 High
IaC Scan	Checkov	Échec si misconfiguration
Build Image	Docker build	Échec si build échoue
Container Scan	Trivy	Échec si CVE Critical
Terraform Plan	Validation infrastructure AWS	Échec si plan invalide
Terraform Apply	Déploiement sur AWS ECS	Uniquement si gates passent
DAST	OWASP ZAP	Échec si $\geq$ 1 High
Remediation	Auto PR	Création branche corrective
Dashboard Update	**Mise à jour interface**	**Visualisation temps réel**

FIGURE 2 – Étapes du pipeline avec security gates, Cloud AWS et IA

3.2 Architecture Globale

Architecture DevSecOps Cloud-Native	
Composant	**Technologie**
CI/CD Orchestration	GitHub Actions
Code Quality & SAST	SonarQube + Semgrep
Secret Detection	TruffleHog
SCA	Snyk / OWASP Dependency-Check
IaC Scanning	Checkov
Container Security	Trivy
DAST	OWASP ZAP
Cloud Provider	AWS (ECR, ECS Fargate, RDS)
IaC Tool	Terraform
AI Model Training	scikit-learn / XGBoost
AI Inference API	FastAPI
Explainability	SHAP / LIME
Frontend Dashboard	**React + Tailwind CSS**
Backend API	**FastAPI**
Database	**SQLite / PostgreSQL**
Reporting	Allure + JUnit

FIGURE 3 – Architecture technique complète

3.3 Matrice des Outils et Technologies (Version Complète)

Étape	Outil(s)	Mode d'exécution
Secret Detection	TruffleHog	GitHub Action (scan commits)
SAST	Semgrep + SonarQube	Docker container + GitHub Action
AI Classification	FastAPI + scikit-learn	Microservice Docker
SCA	Snyk / Dependency-Check	CLI dans pipeline
IaC Scanning	Checkov	Scan Dockerfile et Terraform
Container Scanning	Trivy	Scan image Docker post-build
Cloud Provisioning	Terraform	CLI dans pipeline
Container Registry	AWS ECR	AWS CLI
Orchestration	AWS ECS Fargate	Terraform
DAST	OWASP ZAP	Docker avec automation script (Python)
Backend API	**FastAPI**	**Docker container**
Frontend	**React + Recharts**	**Static build / Docker**
Reporting	Allure + JUnit	Génération automatique
Remédiation	GitHub API	Création automatique de PRs

4 Exigences Fonctionnelles Détaillées

4.1 Application Cible (Vulnerable Web App)

ID	Exigence	Critère d'acceptation
APP-01	Déployer OWASP Juice Shop en conteneur	Application accessible sur `http://localhost:3000`
APP-02	Injecter 3 vulnérabilités supplémentaires	Semgrep doit les détecter avec règles personnalisées

TABLE 3 – Exigences - Application cible

4.2 Pipeline CI/CD

ID	Exigence	Critère d'acceptation
PIP-01	Pipeline déclenché sur `push` et `pull_request`	Workflow YAML valide
PIP-02	Exécution parallèle des jobs indépendants	Temps total $<$ 8 minutes
PIP-03	Cache des dépendances et outils	Réduction de 40% du temps d'exécution

TABLE 4 – Exigences - Pipeline CI/CD

4.3 SAST (Semgrep + SonarQube)

ID	Exigence	Seuil d'échec
SAST-01	Scanner avec règles Semgrep standards + 3 règles personnalisées	$\geq$ 1 Critical
SAST-02	SonarQube analyse avec Quality Gate	Quality Gate Red
SAST-03	Génération rapport SARIF pour GitHub Security tab	Fichier `.sarif` produit

TABLE 5 – Exigences - SAST

4.4 Secret Detection (TruffleHog)

ID	Exigence	Seuil d'échec
SEC-01	Scanner tous les commits (y compris historique)	Tout secret validé
SEC-02	Bloquer le push si secret détecté	Exit code non nul

TABLE 6 – Exigences - Secret Detection

4.5 SCA (Snyk)

ID	Exigence	Seuil d'échec
SCA-01	Scanner `package.json` / `requirements.txt`	$\geq$ 1 Critical ou $\geq$ 3 High
SCA-02	Suggérer versions corrigées dans le rapport	Rapport inclut "Fix version"

TABLE 7 – Exigences - SCA

4.6 Container Scanning (Trivy)

ID	Exigence	Seuil d'échec
CTR-01	Scanner image Docker après build	$\geq$ 1 Critical (CVE)
CTR-02	Scanner les couches de l'image	Rapport détaillé par couche

TABLE 8 – Exigences - Container Scanning

4.7 DAST (OWASP ZAP)

ID	Exigence	Seuil d'échec
DAST-01	Scanner actif sur application déployée	$\geq$ 1 Alert High
DAST-02	Automatisation via `zap-full-scan.py`	Script Python exécutable
DAST-03	Exclusion des faux positifs (baseline)	Fichier `zap-rules.conf`

TABLE 9 – Exigences - DAST

4.8 Modèle IA pour la Sécurité

ID	Exigence	Critère d'acceptation
AI-01	Intégrer un modèle de classification (ex : Random Forest)	Modèle chargé dans un conteneur Python et appelé via API REST
AI-02	Réduire les faux positifs SAST	Le modèle filtre au moins 30% des alertes Semgrep étiquetées comme "bruit"
AI-03	Prioriser les vulnérabilités	Le modèle assigne un score de risque (0-1) basé sur le contexte
AI-04	Auto-apprentissage (optionnel)	Le modèle se réentraîne hebdomadairement avec les nouvelles PR validées
AI-05	Générer un rapport d'explicabilité	Utiliser SHAP ou LIME pour expliquer pourquoi une vuln est classée critique ou non

TABLE 10 – Exigences - Modèle IA

4.9 Déploiement Cloud AWS

ID	Exigence	Critère d'acceptation
CLOUD-01	Provisionner l'infrastructure AWS avec Terraform	`terraform apply` crée VPC, ECS, RDS, ECR sans erreur
CLOUD-02	Scanner les fichiers Terraform avec Checkov	Aucune misconfiguration HIGH
CLOUD-03	Déployer l'image Docker sur AWS ECR après scan Trivy	L'image est push uniquement si Trivy passe
CLOUD-04	Lancer l'application sur ECS Fargate	L'URL publique (Load Balancer) renvoie la page Juice Shop
CLOUD-05	Pipeline doit pouvoir détruire l'environnement	`terraform destroy` dans un job dédié

TABLE 11 – Exigences - Déploiement Cloud

4.10 Interface Dashboard

ID	Exigence	Critère d'acceptation
DASH-01	Dashboard React avec vue d'ensemble du pipeline	Affichage du statut de chaque security gate
DASH-02	Tableau des vulnérabilités avec filtrage	Filtrage par sévérité, outil, statut IA
DASH-03	Visualisation des décisions IA	Affichage du score IA et bouton "Explain" avec graphique SHAP
DASH-04	Panneau de remédiation avant/après	Comparaison visuelle des vulns corrigées
DASH-05	Statut du cloud AWS	Affichage de l'état ECS, RDS, dernière URL déployée
DASH-06	API Backend FastAPI	Endpoints REST pour récupérer les résultats des scans

TABLE 12 – Exigences - Dashboard Interface

4.11 Phase de Remédiation

ID	Exigence	Critère d'acceptation
REM-01	Création automatique d'une PR corrective pour SCA	PR créée avec mise à jour
REM-02	AI-powered fix suggestion	La PR inclut un commentaire généré par l'IA expliquant le correctif
REM-03	Re-scan après correction	Pipeline relancé, seuils validés
REM-04	Dashboard comparatif (avant/après)	Graphique montrant réduction

TABLE 13 – Exigences - Remédiation

4.12 Reporting

ID	Exigence	Critère d'acceptation
REP-01	Génération rapport JUnit pour chaque étape	Intégrable dans GitHub Actions
REP-02	Dashboard Allure consolidé	Vue HTML unique
REP-03	Badge de statut dans README	`![Security Score]`

TABLE 14 – Exigences - Reporting

5 Exigences Non-Fonctionnelles

Catégorie	Exigence	Métrique
Performance	Temps total pipeline (build + scans)	$<$ 8 minutes
Performance IA	Temps d'inférence du modèle	$<$ 500ms par vulnérabilité
Fiabilité	Taux de faux positifs SAST/DAST	$<$ 5%
Coût Cloud	Coût mensuel AWS	$<$ 15€ (free-tier ou ressources minimales)
Maintenabilité	Configuration as code (YAML)	Documentation inline
Extensibilité	Ajout d'un nouvel outil	$<$ 2 heures
Sécurité	Secrets non exposés dans logs	Audit des outputs
Sécurité Cloud	Secrets AWS jamais exposés	Audit des outputs Terraform

6 Livrables Attendus

ID	Livrable	Format	Destinataire
L1	Dépôt GitHub complet	URL publique	Encadrant + Jury
L2	Workflow YAML	`.github/workflows/devsecops.yml`	Encadrant
L3	3 règles Semgrep personnalisées	`.semgrep/custom-rules/`	Jury
L4	Script d'automatisation ZAP	`scripts/zap-scan.py`	Encadrant
L5	Configuration Trivy + Checkov	`.trivyignore`, `.checkov.yaml`	Encadrant
L6	Code Terraform AWS	`terraform/` dossier	Jury
L7	API d'inférence IA (modèle + endpoint)	`ai-service/` (Dockerfile + model.pkl)	Encadrant
L8	Script d'entraînement / fine-tuning	`ai-training/train.py`	Encadrant
L9	**Dashboard React + FastAPI**	`dashboard/` **dossier**	**Jury**
L10	Rapport de projet (50-70 pages)	PDF + DOCX	Jury
L11	Présentation soutenance (15-20 slides)	PowerPoint/PDF	Jury
L12	Démonstration vidéo (7 min)	MP4	Encadrant
L13	Preuve de déploiement cloud (URL)	Dans le rapport	Jury

7 Planning Prévisionnel

Phase	Durée	Activités clés
Phase 1 : Setup	Semaine 1	Application cible, dépôt GitHub, workflow basique
Phase 2 : SAST + Secrets	Semaine 2	Semgrep, règles personnalisées, TruffleHog
Phase 3 : SCA + Container	Semaine 3	Snyk, Trivy, Checkov
Phase 4 : DAST	Semaine 4	OWASP ZAP automation, gestion faux positifs
Phase 5 : Cloud AWS	Semaine 5	Terraform, ECR, ECS Fargate, RDS
Phase 6 : IA Model	Semaine 6	Entraînement modèle, API FastAPI, intégration pipeline
Phase 7 : Dashboard	**Semaine 7**	**React frontend, Backend API, visualisation**
Phase 8 : Remédiation	Semaine 8	PR automatique, re-scan, dashboard comparatif
Phase 9 : Finalisation	Semaine 9	Documentation, rapport, vidéo, soutenance

TABLE 17 – Planning sur 9 semaines (version complète)

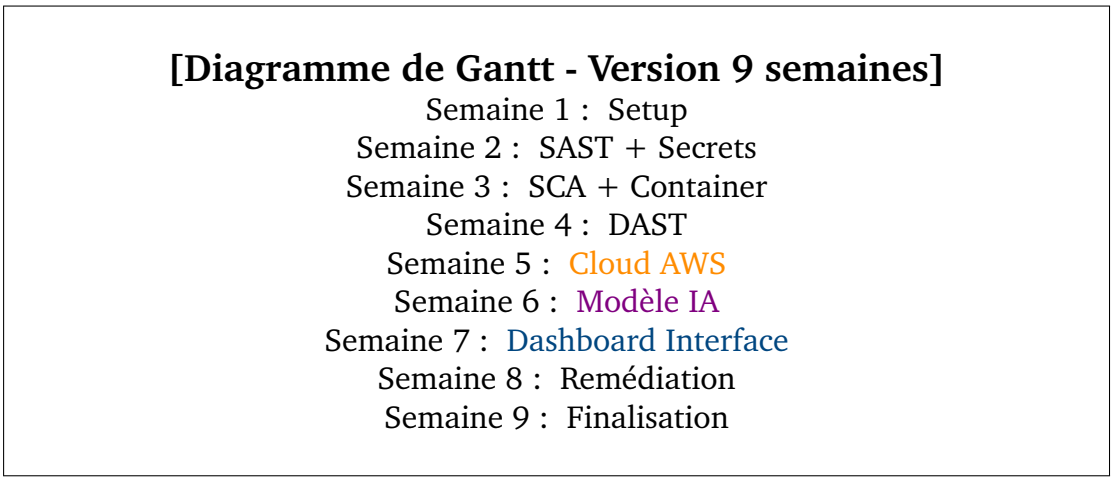


FIGURE 4 – Diagramme de Gantt prévisionnel (version étendue)

8 Critères d'Évaluation

Critère	Pondération	Description
Pipeline complet (7+ étapes)	20%	Tous les scans implémentés et fonctionnels
Déploiement Cloud AWS	15%	Infrastructure as Code, déploiement réussi, sécurité des secrets
Intégration IA	10%	Réduction des faux positifs démontrée, explicabilité SHAP/LIME
Dashboard Interface	**10%**	**Vue claire, interactive, métriques temps réel**
Gestion des faux positifs	10%	Stratégie documentée ET assistée par IA
Optimisation performance	10%	Pipeline $<$ 8 min, parallélisation efficace
Remédiation automatisée	10%	PR automatique et validation post-fix
Qualité de la documentation	10%	Cahier des charges, README, commentaires
Originalité (règles Semgrep)	5%	Règles non triviales, bien justifiées

TABLE 18 – Grille d'évaluation (version avec Cloud, IA & Dashboard)

9 Risques et Mitigations

Risque	Probabilité	Impact	Mitigation
Faux positifs DAST bloquent pipeline	Élevée	Moyen	Configurer `zap-rules.conf` avec seuils ajustables
Temps d'exécution $>$ 8 min	Moyenne	Moyen	Parallélisation, cache Docker layers
Snyk rate limit (API)	Faible	Faible	Utiliser Dependency-Check en fallback
Remédiation automatique casse l'application	Moyenne	Élevé	Tester la PR avec pipeline avant merge
Coût AWS imprévu	Moyenne	Faible	Budget alerts, détruire ressources après démo
Modèle IA trop lent ou inexact	Moyenne	Moyen	Petit modèle + cache ; fallback règles manuelles
Dashboard trop lourd	**Faible**	**Moyen**	**Static generation, lazy loading**

10 Approbations

Rôle	Nom	Date	Signature
Étudiant (PFE)	Votre Nom	__/__/2026	
Encadrant académique	Dr. Nom	__/__/2026	
Encadrant industriel	M. Nom	__/__/2026	

TABLE 20 – Approbations

11 Références Bibliographiques

[11] OWASP Foundation. (2024). *OWASP DevSecOps Guideline*. https://owasp.org/www-project-devsecops-guideline/

[11] GitHub. (2024). *Security hardening for GitHub Actions*. https://docs.github.com/en/actions/security-guides

[11] SonarSource. (2024). *SonarQube Documentation*. https://docs.sonarqube.org/

[11] Semgrep. (2024). *Semgrep Rules Documentation*. https://semgrep.dev/docs/

[11] OWASP ZAP. (2024). *ZAP API Documentation*. https://www.zaproxy.org/docs/api/

[11] Aqua Security. (2024). *Trivy Documentation*. https://aquasecurity.github.io/trivy/

[11] Bridgecrew. (2024). *Checkov Documentation*. https://www.checkov.io/

[11] Truffle Security. (2024). *TruffleHog Documentation*. https://trufflesecurity.com/docs/trufflehog/

[11] AWS. (2024). *ECS Fargate Documentation*. https://docs.aws.amazon.com/ecs/

[11] Hashicorp. (2024). *Terraform Documentation*. https://www.terraform.io/docs/

[11] scikit-learn. (2024). *Machine Learning in Python*. https://scikit-learn.org/

[11] SHAP. (2024). *Explainable AI*. https://shap.readthedocs.io/

[11] **React. (2024). *React Documentation*.** https://react.dev/

[11] **FastAPI. (2024). *FastAPI Documentation*.** https://fastapi.tiangolo.com/

A Annexes

A.1 Glossaire

Terme	Définition
SAST	Static Application Security Testing - Analyse de code source statique

DAST	Dynamic Application Security Testing - Tests dynamiques sur appli en cours d'exécution
SCA	Software Composition Analysis - Analyse des dépendances open source
IaC	Infrastructure as Code - Gestion de l'infrastructure via code (Terraform, CloudFormation)
CI/CD	Continuous Integration / Continuous Deployment
ECS	Elastic Container Service - Orchestration de conteneurs AWS
ECR	Elastic Container Registry - Registry Docker AWS
SHAP	SHapley Additive exPlanations - Explicabilité des modèles IA
LIME	Local Interpretable Model-agnostic Explanations

A.2 Acronymes

— **PFE** : Projet de Fin d'Études
— **CVE** : Common Vulnerabilities and Exposures
— **PR** : Pull Request
— **YAML** : YAML Ain't Markup Language
— **API** : Application Programming Interface
— **AWS** : Amazon Web Services
— **ML** : Machine Learning

A.3 Versioning du Document

Version	Date	Auteur	Modifications
0.1	01/04/2026	Étudiant	Version initiale
0.2	07/04/2026	Étudiant	Ajout sections IaC, Container, Secrets
1.0	14/04/2026	Étudiant	Version finale validée
2.0	**15/04/2026**	**Étudiant**	**Ajout Cloud AWS, Modèle IA, Dashboard Interface**

TABLE 22 – Historique des versions

A.4 Arborescence du Projet

```
pfe-devsecops-pipeline/
 .github/workflows/
    devsecops-pipeline.yml
 app/                              # Application cible
 terraform/                        # AWS Infrastructure as Code
 ai-service/                       # Modèle IA + API FastAPI
    model/
    inference/
```

```
   Dockerfile
dashboard/                      # Interface React + Backend
   frontend/ (React)
   backend/ (FastAPI)
security-tools/                 # Configurations SAST/DAST/SCA
scripts/                        # Automation scripts
reports/                        # Résultats et dashboards
```